Batch 1275 — Completion Addendum (Read-Only)

Scope. Read-only completion record for the ten-company worker-isolation acceptance run (batch 1275). No code, migration, replica, tuning, or rerun was performed to produce this document. Worker-isolation (`WORKER_ISOLATION_ENABLED=true`) remains accepted and enabled on the unchanged build.

Build under test. `b676c2ddf37512dad0e18c6e812a4dc7e58a12a2` (worker service). Confirmed as deployed: `batch_runs.deployment_fingerprint` records `sourceSha = liveAppSha = liveWorkerSha = b676c2d…` (all three match).

Configuration observed. Workspace 3, framework 18, `list_id = NULL` (matches baseline 1274). Two worker replicas × concurrency 4 = 8 execution slots (confirmed empirically below). LLM permit pool `max = 8` .

Evidence sources. Production Postgres (queried read-only via the public proxy) and Railway worker `deploymentLogs` for the active serving deployment `cf79e5a7-baef-4601-a560-989f6c1f3ce2` . Every figure below traces to one of these.

> ***Reliability vs completion are reported separately.*** *Section 2 (completeness, provenance, snapshot, costs, timing) describes pipeline completion. Section 3 (child/permit settlement, parent memory) describes isolation reliability. They are not combined into a single verdict.*

---

1. Headline

Item	Value	Source
Batch status	`completed`	`batch_runs.status`
Jobs	10 total / 10 completed / 0 failed	`batch_runs.total_jobs,completed_jobs,failed_jobs`
Wall clock	00:15:06.706 → 00:39:38.296 UTC = **24m 31.6s**	`batch_runs.started_at,completed_at`
Expected measure results	36 per company × 10 = 360	`framework_measures` (fw 18) × roster
Measure results written	**360 / 360**, 0 null, 0 abstained	`measure_scores`
Snapshot saved	yes (`snapshot_saved = true` ; snapshot row immutable)	`batch_runs` , `analysis_results`
Recorded LLM cost	**$4.0532** over 14,027,178 tokens	`llm_usage_events`
Isolation warnings in logs	**0** (no quarantine / forced-kill / withheld-permit lines)	worker `deploymentLogs`

No stale or incomplete pipeline output was found. The only explicit flags raised are provenance/telemetry gaps, enumerated in Section 4 — none of them indicates a missing or stale measure result.

---

2. Completion evidence

2.1 Measure completeness — 36 expected per company

Framework 18 defines exactly **36** measures. `measure_scores` for batch 1275 holds **360 rows** = 10 companies × 36 distinct `measure_id` , with **0 null scores** and **0 abstentions**. Every company produced a full, scored 36/36. “No information” outcomes, had any occurred, would remain legitimate scored results; none were recorded as null/abstained here.

2.2 Batch & attempt provenance

All 10 jobs completed on **attempt 1** (no retries). The pipeline attempt id recorded in `measure_scores.attempt_id` for each company is identical across all 36 of that company’s rows, and matches the `attempt_id` in the worker log’s `[DIAG][job-transition] … -> completed` line for the same job — an end-to-end log→DB provenance match:

Job	Company	Attempts	Pipeline `attempt_id` (log = DB)
96905	147 Adani Enterprises	1	bcc25b41-…-f13cd4a4ebc9
96906	148 Mapfre	1	3608cbed-…-bc4878566a0e
96907	158 3i Group	1	3e4caacc-…-04394c9d0897
96908	257 Monster Beverage	1	6add74c3-…-bb9e618d5af0
96909	1063 Emaar	1	074f414f-…-847a2ffe0f21
96910	1486 Sumitomo Metal Mining	1	2384f321-…-9bea478fce (2384f321-8240-49e6-8a60-2b9eca478fce)
96911	1533 Teva	1	0cbf8be4-…-f063c37c3d71
96912	2004 Yangtze Optical	1	c5559e56-…-01f50da1d1d7
96913	2682 Netflix	1	c6d501dd-…-07f724b70c69
96914	2696 ExxonMobil	1	e06707f4-…-021e185363ac

Note on identifier spaces: `analysis_jobs.attempt_token` (job-queue claim token) and `measure_scores.attempt_id` / log `attempt_id` (pipeline attempt id) are **distinct UUID namespaces** — this is by design, not a defect. The pipeline `attempt_id` is the one that ties scored results to the executing attempt, and it is internally consistent across all 360 rows.

2.3 Saved snapshot state

`batch_runs.snapshot_saved = true`. The snapshot row `analysis_results.id = 789` (framework_name “AI Management”, `companies_count = 10`, `average_score = 15`) was written at 00:39:45.468 — 7.2s after batch completion — with `immutable_snapshot = true`. The snapshot is present, immutable, and consistent with the 10-company roster.

2.4 Final child / permit settlement

See Section 3 (isolation reliability). Summary: no quarantine or withheld-permit events; LLM permits returned to `held = 0, waiting = 0` at end of run.

2.5 Recorded costs

`llm_usage_events` for batch 1275: **1,852 events**, first 00:17:36, last 00:39:42.

Metric	Value
Prompt tokens	13,120,586
Completion tokens	906,592
Total tokens	**14,027,178**
Input cost	$3.6725
Output cost	$0.3807
Total cost	**$4.0532**

Per-company cost ranged **$0.3651 – $0.4520** (mean ≈ $0.405). Provider/model mix: deepseek `deepseek-chat` 1,846 events; openrouter `deepseek/deepseek-chat-v3.1` 5 events; openai `gpt-4o` 1 event — i.e. provider fallback fired on 6 of 1,852 calls (0.3%). Observation only; all calls resolved and all jobs completed.

Company	Events	Tokens	Cost (USD)
147 Adani Enterprises	156	1,293,028	0.3726
148 Mapfre	176	1,418,688	0.4121
158 3i Group	176	1,501,951	0.4293
257 Monster Beverage	174	1,320,884	0.3802
1063 Emaar	165	1,370,092	0.3947
1486 Sumitomo Metal Mining	214	1,556,785	0.4520
1533 Teva	157	1,266,363	0.3651
2004 Yangtze Optical	306	1,514,354	0.4464
2682 Netflix	159	1,324,199	0.3814
2696 ExxonMobil	169	1,460,834	0.4194

2.6 Queue-wait vs execution time

All 10 jobs were created at 00:15:06. **8 jobs were claimed immediately (queue wait ≈ 0s)** — exactly the 8 slots from 2 replicas × concurrency 4. **2 jobs queued** until a slot freed, confirming the capacity ceiling:

Job	Company	Queue wait	Execution	Claimed → Completed
96905	147 Adani	0s	9m 34s	00:15:06 → 00:24:40
96906	148 Mapfre	0s	12m 40s	00:15:06 → 00:27:47
96907	158 3i Group	0s	17m 17s	00:15:06 → 00:32:23
96908	257 Monster	0s	13m 20s	00:15:06 → 00:28:26
96909	1063 Emaar	**9m 34s**	13m 26s	00:24:41 → 00:38:07
96910	1486 Sumitomo	0s	**23m 40s**	00:15:06 → 00:38:46
96911	1533 Teva	0s	12m 18s	00:15:06 → 00:27:24
96912	2004 Yangtze	0s	21m 31s	00:15:06 → 00:36:38
96913	2682 Netflix	0s	12m 18s	00:15:06 → 00:27:24
96914	2696 Exxon-Mobil	**12m 40s**	11m 51s	00:27:47 → 00:39:38

Emaar (96909) was claimed the instant Adani (96905) freed a slot (00:24:40 → 00:24:41); ExxonMobil (96914) likewise when Mapfre (96906) freed one (00:27:47). The two queued jobs account for the long tail: the batch finished when the last queued job (ExxonMobil) completed at 00:39:38.

---

3. Isolation reliability (reported separately from completion)

3.1 Child / permit settlement — clean, with one retention caveat

The isolation layer (`server/lib/job-isolation.ts`) emits `[WORKER-ISOLATION]` log lines **only on abnormal paths**: quarantine, grace-period SIGTERM/SIGKILL of child process groups, late/discarded child reports, and withheld permits. In the retained worker log window (00:21:10 → 01:02:01 UTC, which covers the first job completion at 00:24:40 through to end-of-run and well beyond), there are:

- **0** `[WORKER-ISOLATION]` lines

- **0** quarantine events
- **0** forced-kill (SIGTERM/SIGKILL) events
- **0** late-report / withheld-permit events

LLM permit accounting (`[DIAG][llm-permits]` , 326 samples): pool `max = 8` ; peak `held = 8` (fully saturated under load); peak `waiting = 10` (queued demand behind the 8-slot ceiling); and the run **settled to `held = 0, waiting = 0`** by end. No permit leak.

Combined with all 10 jobs completing on attempt 1, this is consistent with every child process having forked, executed, exited cleanly, and returned its permits — i.e. clean child/permit settlement.

FLAG (evidence boundary, not a failure). This is an *inference from the absence of trouble markers plus the permit return*, not a per-child fork/exit trace. Two reasons: (a) a clean child exit is deliberately silent in this build — only problems log — so there is no positive per-child "forked/exited" line to count; and (b) the first ~6 minutes of the run (00:15:06 → 00:21:10) exceeded the log API's 5,000-line-per-window cap and were not retrievable, so any startup-window markers there are unavailable. No fabricated per-child record is supplied in their place.

3.2 Parent process memory vs container/child memory

The event-loop telemetry (`[DIAG][event-loop] role=worker` , 326 samples) reports **`rss_mb` of the parent worker process only** (role `worker` = the long-lived worker-main parent; no `role=primary` / child event-loop samples are emitted):

- Parent RSS steady state: ~138–194 MB (mean 164 MB, p95 193 MB), event-loop lag p99 ≈ 0–1 ms throughout the working run.

FLAG (labelling). These `rss_mb` values are **parent-process RSS**. Total container memory and per-child RSS are **not separately instrumented** in the retained telemetry, so no container-level or child-level memory figure is asserted here. Do not read the parent RSS as a container total.

3.3 Completion-time event-loop spike — retained as observation, not diagnosis

One sample stands out, 12s after batch completion:

```
00:39:50.692  [DIAG][event-loop] role=worker  max_lag_ms=579  rss_mb=855
```

Parent RSS jumped from the ~150 MB baseline to **855 MB**, and event-loop `max_lag_ms` to **579 ms** (vs 0–9 ms for the entire working run). RSS held at 855 MB for ~30s (00:39:50 → 00:40:20) and fell back to ~145–189 MB by 00:40:39.

This coincides with post-batch finalisation (snapshot write at 00:39:45, cost-event flush to 00:39:42). It is **retained as an observation only**. No mechanism is asserted: it is not established whether the spike is snapshot serialisation, a last child's teardown, GC pressure, or log flushing. It did not prevent completion, did not trip isolation, and resolved on its own. Treat as a candidate to instrument in the 25-company run, not as a proven cause.

3.4 Browser-render subprocess pressure (observation)

Between ~00:24:22 and ~00:25:35 the logs show repeated `/usr/bin/chromium: Cannot fork` and `posix_spawn … Resource temporarily unavailable (11)` from the headless-browser render path. These are **browser subprocess** limits during page rendering — distinct from the job-isolation child processes — and did not fail any job (all affected companies completed with success). Observation only; a candidate to watch at larger scale where browser-render concurrency rises.

4. Explicit flags (open items — none blocks acceptance)

1. `analysis_jobs.worker_id` **is blank** for all 10 jobs. Worker/replica attribution is therefore not available from that column; slot/replica behaviour was reconstructed from claim timing (Section 2.6). Minor provenance gap.
2. `analysis_results.deployment_fingerprint` **is empty** on snapshot 789, whereas `batch_runs` carries the full SHA (`b676c2d…`). The snapshot is not self-describing of its build; provenance must be joined via `batch_runs`. Minor provenance gap.
3. **First ~6 min of worker logs not retained** (5,000-line/window API cap). Startup-window markers unavailable — see §3.1. Evidence boundary, not a defect.
4. **Container/child memory not instrumented** — only parent RSS is logged (§3.2). Labelling caveat.
5. `acceptance_state = pending` on both `batch_runs` and `analysis_results`. This is an observed state, not a claim of pass/fail; the acceptance decision is the user’s.
6. **Completion-time spike unexplained** (§3.3) — carried forward as an observation to instrument, not a diagnosis.

5. Bottom line

- **Pipeline completion:** complete and consistent — 360/360 measures, 0 null, 0 abstained, all attempt-1, immutable snapshot saved, $4.0532 recorded. No stale or missing output.
- **Isolation reliability:** no quarantine, no forced kills, no permit leak (permits returned to 0), parent RSS healthy aside from a transient post-completion spike retained as an observation. Child/permit settlement is inferred (clean) with the retention caveat in §3.1, reported separately from completion.